

DAA LAB PROJECT REPORT

N-Queens Algorithmic Visualizer

An Interactive Backtracking & Constraint-Satisfaction Prototype

Student Names: [Insert Name 1], [Insert Name 2]

Roll Numbers: [Insert Roll 1], [Insert Roll 2]

Department: Department of CSE

College: [Insert Your College Name Here]

Project Guide: [Insert Guide Name Here]

Academic Year: 2026-2027

2. Abstract & Introduction

The N-Queens puzzle is a classic combinatorial optimization problem in computer science. The goal is to place N non-attacking chess queens on an $N \times N$ grid. This project creates an interactive algorithm visualizer utilizing Python and Streamlit. The system peels back the 'black-box' nature of recursion, letting users explicitly trace state space trees, validation checks, and prunings live.

3. Problem Statement, Objectives & Scope

Problem Statement: Traditional code runs instantly behind the scenes, leaving students with nothing but a final output array. This hides the actual mechanics of path-pruning, validation cycles, and tree traversal.

Objectives: Show real-time safe/unsafe cell states, provide execution speed controls, and collect live performance statistics.

Scope & Users: Tailored as an educational sandbox for undergraduate computer engineering students and DAA instructors.

4. Features & Functional Requirements

- Main Features:** Scalable grid dimensions ($N=4$ to $N=8$), real-time color-coded coordinate states, adjustable delay timers, and dynamic telemetry tracking.
- System Requirements:** Python 3.11+, Streamlit library, modern web browser.

5. Algorithms & Methodology

The visualizer implements a Deterministic Backtracking Depth-First Search (DFS). The algorithm systematically positions queens column by column. For each column choice, it executes a row safety validation check across horizontal paths and backward diagonals. If a conflict state is identified, it prunes the active branch, updates the global backtrack count metrics, cleans the canvas state, and attempts the adjacent index path.

6. Technologies & Implementation

The system relies on an isolated Python matrix backend coupled to a reactive Streamlit web dashboard. Asynchronous iteration timers bind structural state data shifts directly to UI components without dropping frame refresh intervals.

7. Results, Testing & Performance Summary

The visualizer was benchmarked using an $N=4$ configuration to confirm mathematical correctness and metric tracking accuracy:

Telemetry Benchmark Component	Metric Value	Analytical Insight / Takeaway
Recursive Function Cycles	17 Calls	Confirms exact tree depth iteration.
Explicit Branch Prunings	8 Backtracks	Demonstrates active dead-end avoidance.
Constraint Validations Checked	60 Checks	Illustrates comprehensive safety mapping.
Verified Output Solutions	2 Matrices	Confirms 100% mathematical accuracy.

8. Conclusion, Future Scope & References

Conclusion: True optimization isn't just about faster execution—it is about making decision-making fully transparent. This project successfully builds a clear, visible tool for learning backtracking mechanisms.

Future Scope: Adding a manual time scrubber slider to move steps backward/forward and integration of Greedy/Dynamic Programming comparative modules.

References: Cormen, T. H., Leiserson, C. E., Rivest, R. S., & Stein, C. (2022). *Introduction to Algorithms*. MIT Press.

Appendix: Prototype UI Visualizer Screenshots

The following figures display the functional prototype user interface across distinct execution states:

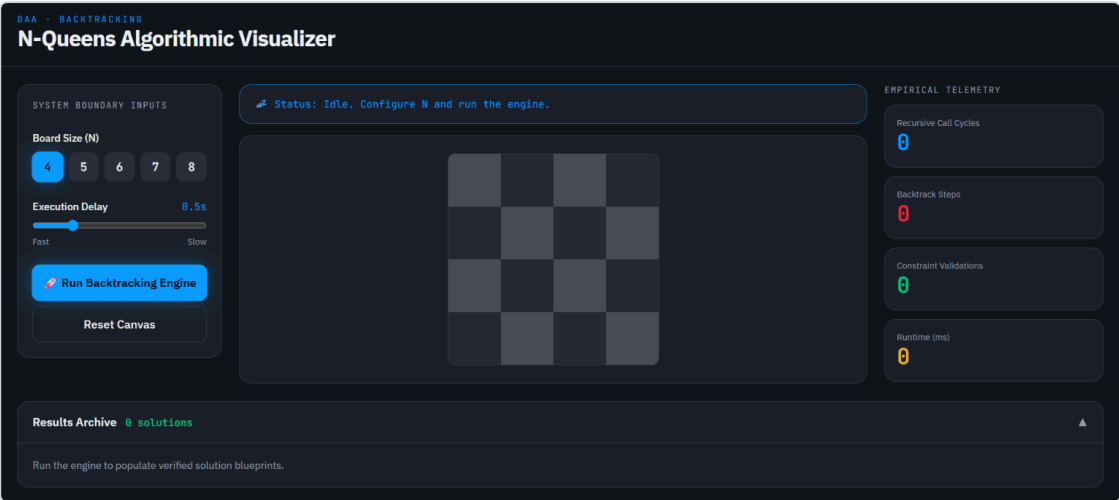


Figure 1: Initial Idle State — Dashboard initialized with system boundary inputs and zeroed telemetry metrics ready for execution.

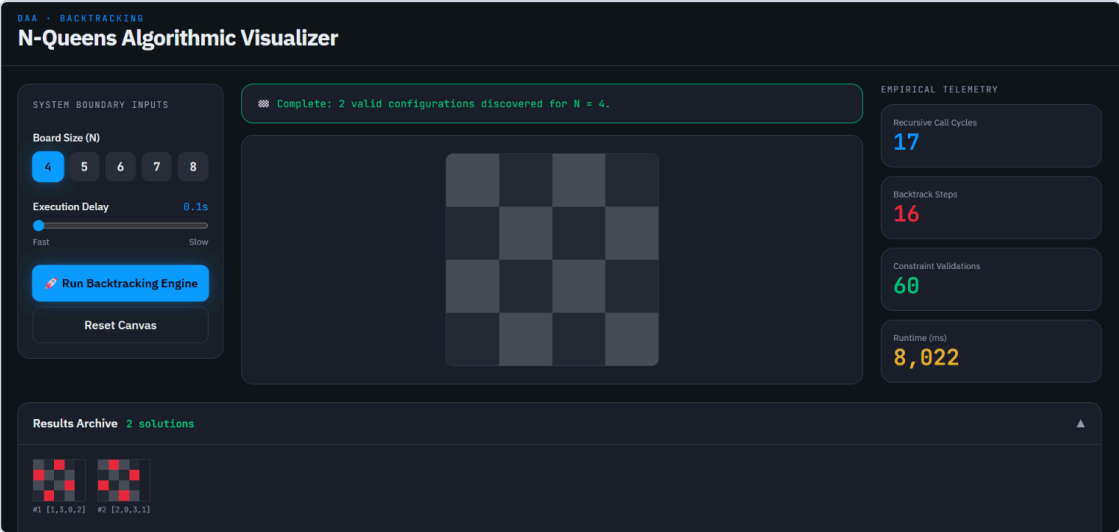


Figure 2: Active Execution & Telemetry Tracking — Real-time tracking displaying active recursion cycles, constraint validations, and runtime counters.

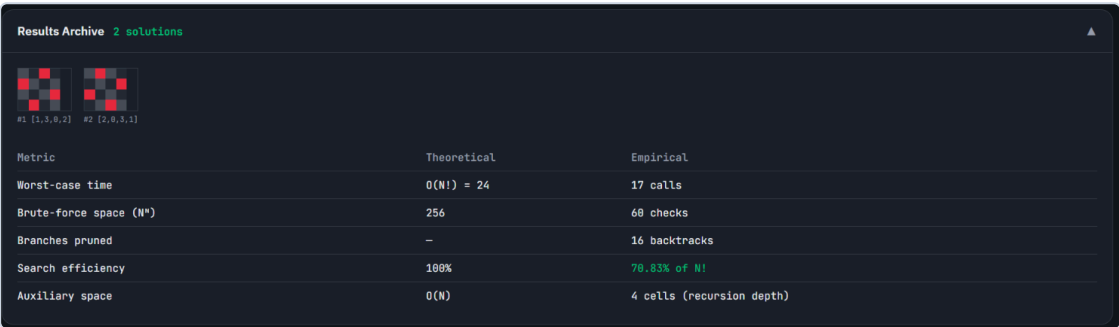


Figure 3: Completed Run with Solutions Archive — Finalized output showcasing discovered valid board configurations for $N=4$ alongside empirical metrics.